

MVP_RUN_STEPS

Helios MVP — Complete Setup Guide (first-time dbt + BigQuery)

This walks you from a blank machine to a **working MVP**: a real, dollar-priced, mix-vs-rate diagnosis of the Google Merchandise Store funnel. No prior dbt or BigQuery experience assumed. Follow the parts **in order**. After each part there's a "✅ You should see" and an "❌ If it fails" so you're never stuck.

Run a checkpoint anytime: `python scripts/preflight.py` prints exactly what's done and what's left. Run it after each part.

0. What these tools are (30-second primer)

- **BigQuery** = Google's cloud data warehouse. It stores big tables and runs SQL on them. Our raw data (`bigquery-public-data.ga4_obfuscated_sample_ecommerce`) already lives there, free and public. We just need a project to run queries *from*.
- **dbt** = a tool that turns a folder of `SELECT` statements into managed BigQuery tables, in dependency order, and runs tests on them. `dbt build` = "build all my tables and test them." That's the whole idea.
- **The flow:** raw GA4 events → dbt builds clean tables (`staging` → `intermediate` → `marts`) → `diagnose.py` reads the marts and writes a Decision Brief.

Cost: free tier = 1 TB queries/month. This project scans < 1 GB/run, capped at 5 GB. Expect ≈ \$0. Billing is required only to switch the BigQuery API on.

Part A — Get a dbt-compatible Python (3.9–3.12) (~0–5 min)

Our pinned dbt (1.8) supports Python **3.9 through 3.12** — but NOT 3.13/3.14. Check what you already have:

```
py --list
```

- If you see **3.12** (or 3.9/3.10/3.11) → you're done, use that. We'll use **3.12**.
- If you only have 3.13/3.14 → install 3.12 from <https://www.python.org/downloads/release/python-31210/> ("Windows installer 64-bit", **tick "Add python.exe to PATH"**), then re-run `py --list`.

Everywhere below, `py -3.12` selects that interpreter regardless of your default.

✓ `py -3.12 --version` prints `Python 3.12.x`.

Part B — Install the Google Cloud SDK (gcloud) (~5 min)

This lets dbt log in to BigQuery as you.

1. Download: <https://cloud.google.com/sdk/docs/install> → "Windows" → run the installer.
2. Accept defaults. At the end leave "Run gcloud init" ticked (or run `gcloud init` later).
3. Open a **new** terminal and check:

```
gcloud --version
```

✓ You should see a list including `Google Cloud SDK 5xx ...`. ✗ If not found, reopen the terminal (the installer edits PATH).

Part C — Create a GCP project with billing (~10 min, one-time)

1. Go to <https://console.cloud.google.com> and sign in with your Google account.
2. **Create the project:** top bar, click the project dropdown (says "Select a project") → **NEW PROJECT** → Name it `helios-mvp` → **CREATE**.
3. **Note your Project ID.** Open the dropdown again — the ID is under the name and may differ (e.g. `helios-mvp-472310`). **You'll paste this everywhere** `<YOUR_PROJECT>` **appears.** (Cloud Console home → "Project info" card also shows it.)
4. **Enable billing:** left menu (☰) → **Billing** → "Link a billing account" → add a payment method if you don't have one. (Free tier covers this project.)
5. **Enable the BigQuery API:** left menu → **APIs & Services** → + **ENABLE APIS AND SERVICES** → search **BigQuery API** → **ENABLE**. (Often already on.)

✓ You now have a project ID, billing linked, BigQuery API enabled. ✗ Common snag: if later steps say "billing not enabled," redo step 4 for *this* project (check the project name in the top bar is `helios-mvp`).

Part D — Point the project at your config (~2 min)

1. Open `profiles.yml` in the repo root (any text editor / VS Code).
2. Find the **two** lines that say:

```
project: helios-analytics    # ← CHANGE to YOUR GCP project id
```

(one under `dev:`, one under `prod:`). Replace `helios-analytics` with **your** Project ID from Part C step 3. Save.

✓ Both `project:` lines now show your real project ID.

Part E — Create the Python environment + install dbt (~5 min)

In the repo root (`C:\Users\anand\helios`):

```
py -3.12 -m venv .venv
.venv\Scripts\activate
pip install -U pip
pip install -r requirements.txt
```

The prompt should now start with `(.venv)`. The install pulls dbt, BigQuery, scipy, etc. (~2–3 min).

✓ `dbt --version` prints `Core: 1.8.x` and `bigquery: 1.8.x`. ✗ If install errors mentioning Python version, you're not in the 3.12 venv — re-run `py -3.12 -m venv .venv` and re-activate.

Every new terminal: re-activate with `.venv\Scripts\activate` before running `dbt / python`.

Part F — Log in to BigQuery (ADC) (~2 min)

```
gcloud auth application-default login
gcloud config set project <YOUR_PROJECT>
```

The first command opens a browser — pick your Google account and click **Allow**.

✓ `gcloud auth application-default print-access-token` prints a long token string. ✗ If the browser step fails, run it again; make sure you allow all requested scopes.

Part G — Prove dbt can reach BigQuery (Milestone M0) (~2 min)

```
dbt debug --profiles-dir .
```

This checks your config + connection. (`--profiles-dir .` tells dbt the `profiles.yml` is here in the repo, not the default home folder.)

✓ You should see a list ending in **All checks passed!** — every line "OK". ✗ Troubleshooting:

- Could not find profile named 'helios' → make sure you included `--profiles-dir .` and you're in the repo root.
- Could not automatically determine credentials → redo Part F.
- Not found: Dataset ... in location ... → confirm `location: US` is set in `profiles.yml` (it is by default) and your project exists.

Part H — Build the whole data spine (Milestones M1–M5) (~3–5 min)

```
dbt deps --profiles-dir .
dbt build --profiles-dir .
```

- `dbt deps` downloads dbt helper packages (one-time).
- `dbt build` creates the tables **in order** and runs **every test**: staging views → the two keystone models → the marts (`fct_funnel` , `fct_daily_funnel` , `fct_orders` , `dim_channels` , `dim_date`) → monotonicity, uniqueness, revenue-reconcile, and the golden unit tests.

It creates BigQuery datasets named like `helios_dev_staging` and `helios_dev_marts` (your tables live there). You can see them at <https://console.cloud.google.com/bigquery>.

✓ Ends with a green summary, e.g. `Completed successfully` and `PASS=NN WARN=.. ERROR=0`.

✗ Troubleshooting:

- `unique_items ... not found` → open `models/marts/finance/fct_orders.sql`, delete the line `ecommerce.unique_items as unique_items`, and the `p.unique_items` line, save, re-run. (Tell me and I'll do it.)
- **A `test_revenue_reconciles` failure** → expected-ish on first run (raw vs deduped revenue). Paste me the numbers and I'll set the right tolerance.
- `require_partition_filter` errors shouldn't happen (already relaxed for dev). If one does, tell me.
- Anything else red → **copy the whole error block to me**; I'll fix it with you.

Part I — Validate the semantic layer + run the math tests (~1 min)

```
python scripts/validate_semantic.py
pytest tests/ -q
```

✓ Validator prints `PASS - 0 dangling refs`. pytest prints all green (decomposition golden + significance).

Part J — 🎉 THE PAYOFF: your first diagnosis

```
python diagnose.py --project <YOUR_PROJECT> --dataset helios_dev_marts
```

✓ Prints a **Decision Brief**: the biggest week-over-week conversion move on the real data, split into **mix-shift vs rate-change**, significance-tested, **priced in dollars**, with a recommended action. **That is the MVP.**

✗ If it says "returned no rows" or "table not found": the dataset name differs. Go to the BigQuery console, find the dataset holding `fct_daily_funnel`, and pass it: `python diagnose.py --project <YOUR_PROJECT> --dataset <that_name>`. (Or tell me.)

One-page command recap (after Parts A–D are done)

```
.venv\Scripts\activate
gcloud auth application-default login
gcloud config set project <YOUR_PROJECT>
dbt debug --profiles-dir .
dbt deps --profiles-dir .
dbt build --profiles-dir .
python scripts/validate_semantic.py
pytest tests/ -q
python diagnose.py --project <YOUR_PROJECT> --dataset helios_dev_marts
```

When to ping me

Paste the output back to me at **any** failing step — especially the first `dbt debug` and `dbt build`. I'll debug live, fix the file, and get you to the brief. You are about 5 commands from a working MVP.